



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica 6

Silverio Martínez Andrés

Lab. Organización y Arquitectura de
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 10 de mayo de 2026

Práctica 6

1. OBJETIVO

Generar de diferentes formas un multiplexor (MUX) por medio del lenguaje VHDL utilizando el software Quartus II

2. INTRODUCCIÓN

En la presente práctica se realizó la implementación de un decodificador binario de 2 a 4 líneas mediante el lenguaje de descripción de hardware VHDL, empleando las herramientas Quartus Prime para la síntesis y ModelSim para la verificación funcional.

Este dispositivo representa un bloque fundamental en la lógica digital, cuya función principal es la selección de una salida específica basada en una combinación de entrada. El desarrollo del proyecto abarcó desde la descripción del circuito (enfoques conductual y estructural) hasta el análisis de su comportamiento lógico.

Se puso especial énfasis en la influencia de la señal de habilitación (enable) y las líneas de dirección, validando mediante simulación temporal la transición entre el estado de alta impedancia o reposo y la activación selectiva de salidas. Esta experiencia permitió consolidar conceptos clave sobre jerarquía de diseño, señales de control y el flujo de trabajo en herramientas CAD.

3. DESARROLLO

1) Crear un proyecto con nombre practica 6, utilizando la misma forma de generarlo conforme a lo realizado en la anterior practica 1

Paso 1: Seleccionamos la opción de “New Project Wizard”

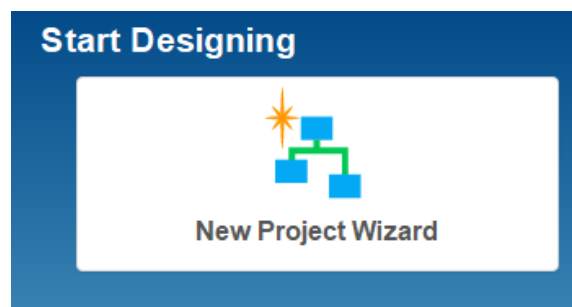


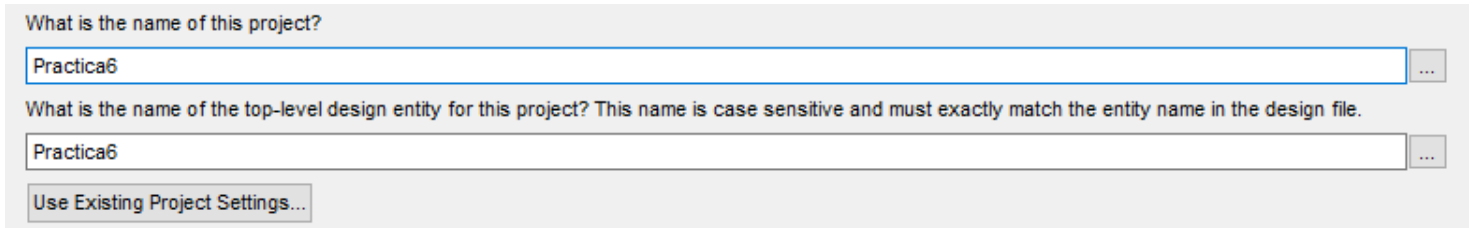
Imagen 1: New Project Wizard

Paso 2: Seleccionamos la carpeta donde queremos guardar el proyecto



Imagen 2: Directory for this project

Paso 3: Le asignamos el nombre del proyecto



What is the name of this project?

Practica6

What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.

Practica6

Use Existing Project Settings...

Imagen 3: Name of this Practica3

Paso 4:



New Project Wizard

Project Type

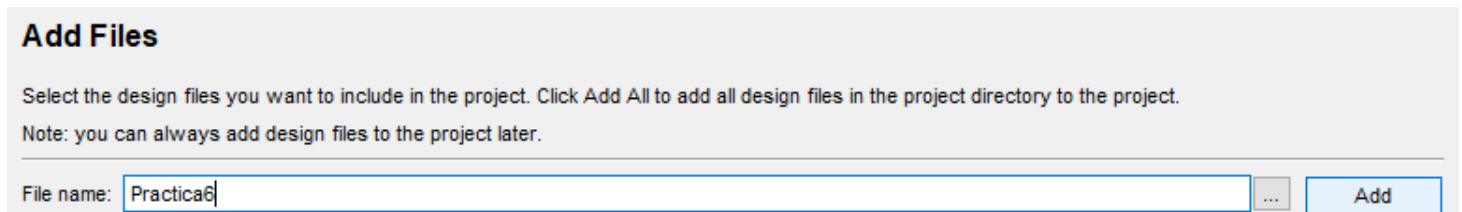
Select the type of project to create.

☒ Empty project

Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.

Imagen 4: Project Type

Paso 5: Añadimos el archivo “Practica3”



Add Files

Select the design files you want to include in the project. Click Add All to add all design files in the project directory to the project.
Note: you can always add design files to the project later.

File name: Practica6

Add

Imagen 5: Add Files

Paso 6: Llegamos a esta ventana y ponemos en auto

Family & Device Settings

Select the family and device you want to target for compilation.

You can install additional device support with the Install Devices command on the Tools menu.

To determine the version of the Quartus Prime software in which your target device is supported, refer to the [Device Support List](#) webpage.

Device family

Family: Cyclone IV E

Devices: All

Target device

☒ Auto device selected by the Filter

☐ Specific device selected in 'Available devices' list

☐ Other: n/a

Show in 'Available devices' list

Package: Any

Pin count: Any

Core Speed grade: Any

Name filter:

☒ Show advanced devices

< Back

Next >

Finish

Cancel

Help

Imagen 6: Family & Device Settings

Paso 7: Para terminar, nos aparecerá la siguiente ventana y le daremos clic en “Finish”

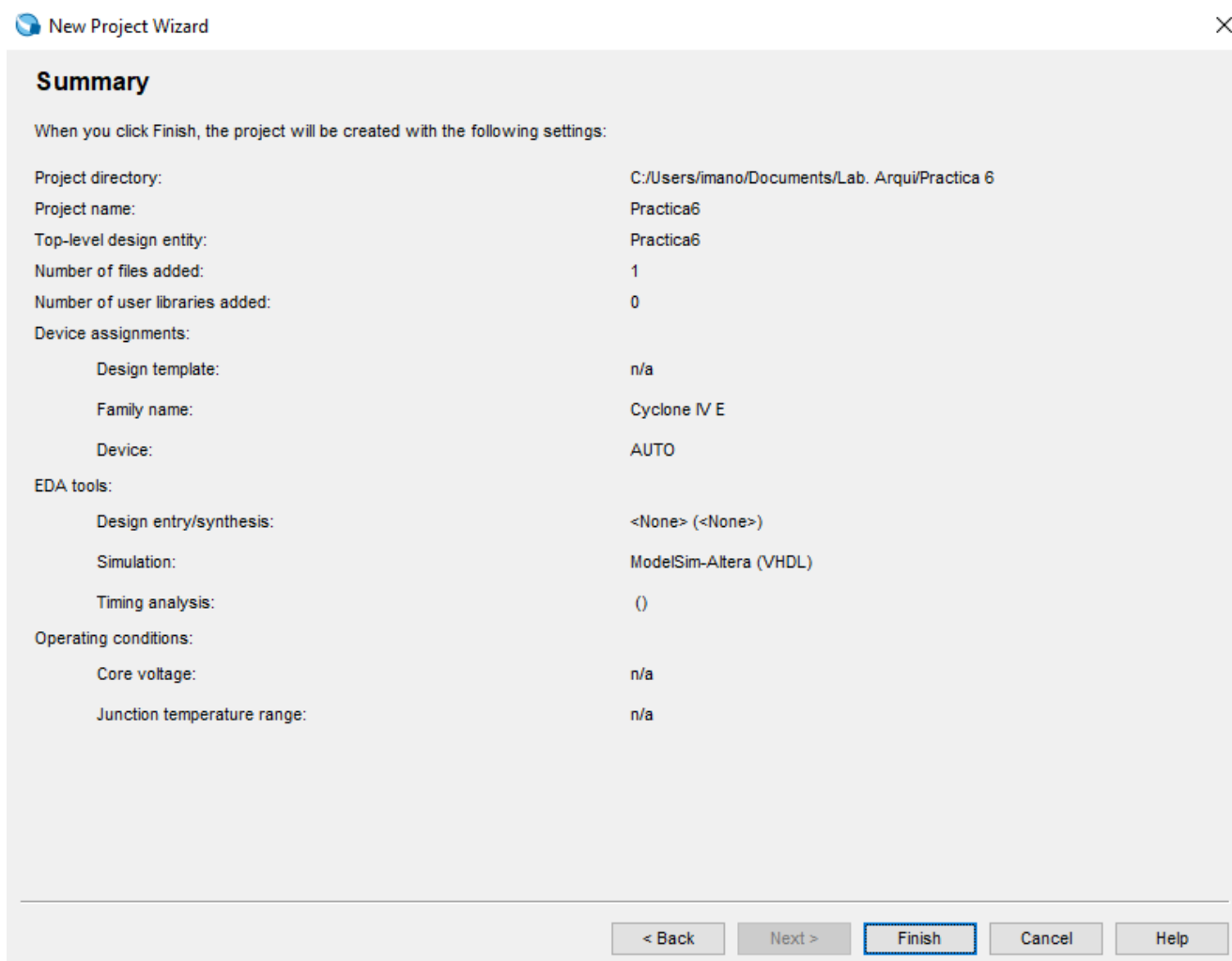


Imagen 7: Summary

2) El tipo de dispositivo a utilizar considere Auto

En la siguiente imagen vemos, que pusimos Auto en la configuración:

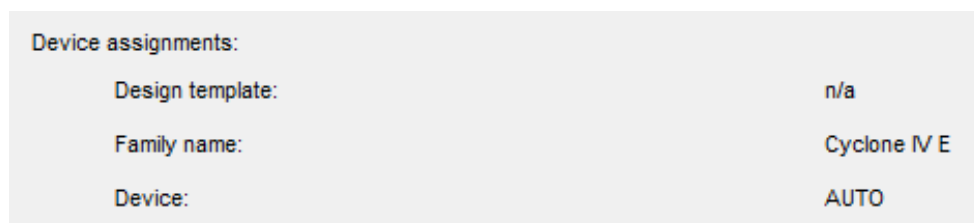


Imagen 8: Device AUTO

3) Crear un archivo nuevo de diseño de tipo VHDL

Paso 1: Damos clic en “File” y después en “New”

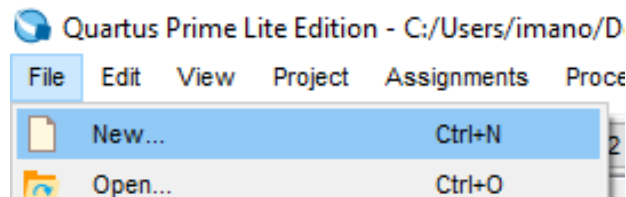


Imagen 9: Nuevo archivo

Paso 2: En la siguiente ventana, dar clic en “VHDL File” y después clic en “Ok”

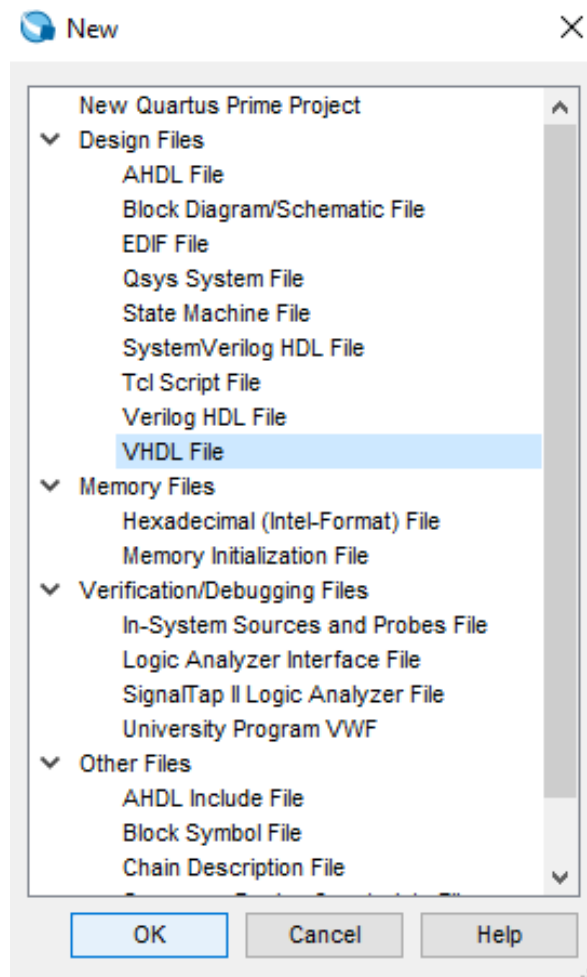


Imagen 10: VHDL File

4) Una vez que este generado teclear en el editor el código

```
library ieee;
use ieee.std_logic_1164.all;

entity practica6 is
port (A: in std_logic_vector(1 downto 0);
      E: in std_logic;
      O: out std_logic_vector(3 downto 0)); --- salidas
end practica6;

architecture DEC of practica6 is
begin
process (A,E)
begin
if (E = '0') then
O<= "0000";
else
case A is
when "00" => O<= "0001";
when "01" => O<= "0010";
when "10" => O<= "0100";
when "11" => O<= "1000";
when others => O<= "0000";
end case;
end if;
end process;
end DEC;
```

Imagen 11: código

Este diseño recibe **2 bits de entrada (A1 y A0)** y una señal de **habilitación (E)**. Su salida es un vector de **4 bits (O3, O2, O1, O0)**.

Cuando la entrada **E = '1'**, el decodificador **activa (pone en 1)** solo una de las salidas según el valor binario de **A**.

Cuando **E = '0'**, todas las salidas son **"0000"** (desactivadas).

3. Interpretación práctica

Esto es un **decodificador binario**, muy usado en:

- Selección de líneas en memorias o multiplexores.
- Activar solo una salida entre varias posibles según una dirección binaria.
- Implementar sistemas de control donde solo un dispositivo puede estar activo a la vez.

4. Ejemplo

Supongamos:

- E = '1'
- A = "10" (2 en decimal)

Entonces la salida sería:

- O = "0100" → activa la **tercera salida (O2)**.

Si después E = '0', sin importar A, todas las salidas vuelven a 0000.

Resumen final

1. **Tipo de circuito:** Decodificador 2 a 4
2. **Entradas:** A(1:0), E
3. **Salidas:** O(3:0)
4. **Función:** Activa una única salida según la dirección de entrada si E=1.
5. **Uso típico:** Selección de líneas, direccionamiento, control lógico.

5) Compilar el programa Revisar que la compilación fue de manera exitosa

Como vemos en la siguiente imagen, se muestra que fue exitosa la compilación:

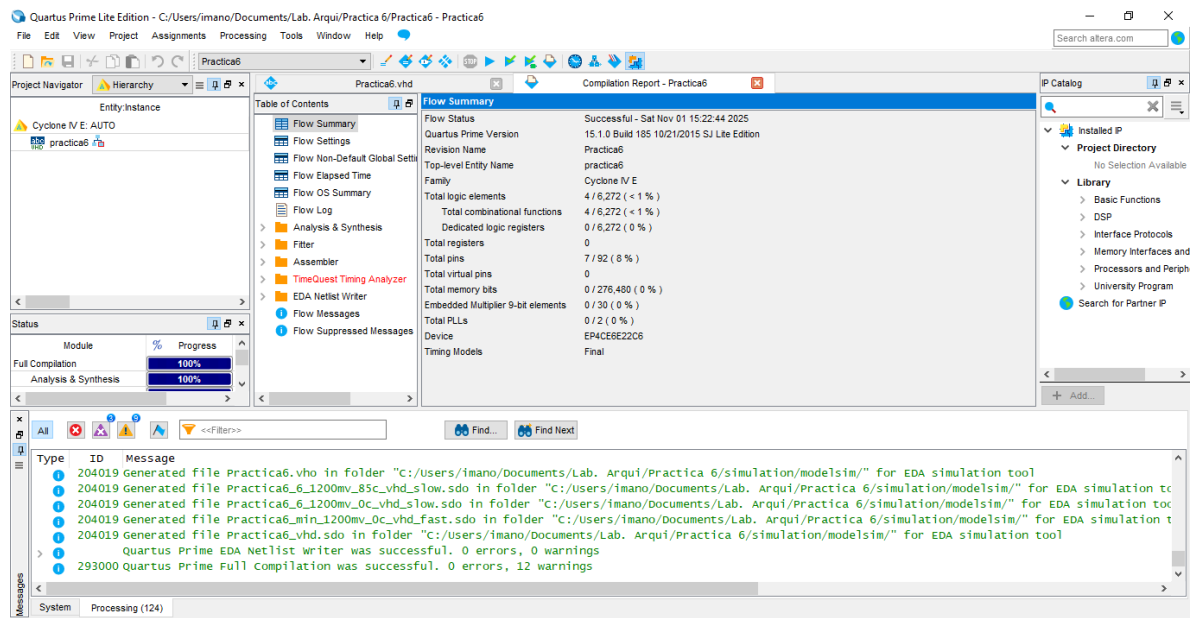


Imagen 12: Compilación

6) Procedemos a revisar el circuito sintetizado (RTL Viewer)

Paso 1: Nos dirigimos en la pestaña de “Tools”, buscamos “Netlist Viewers” y finalmente damos clic en “RTL Viewer”

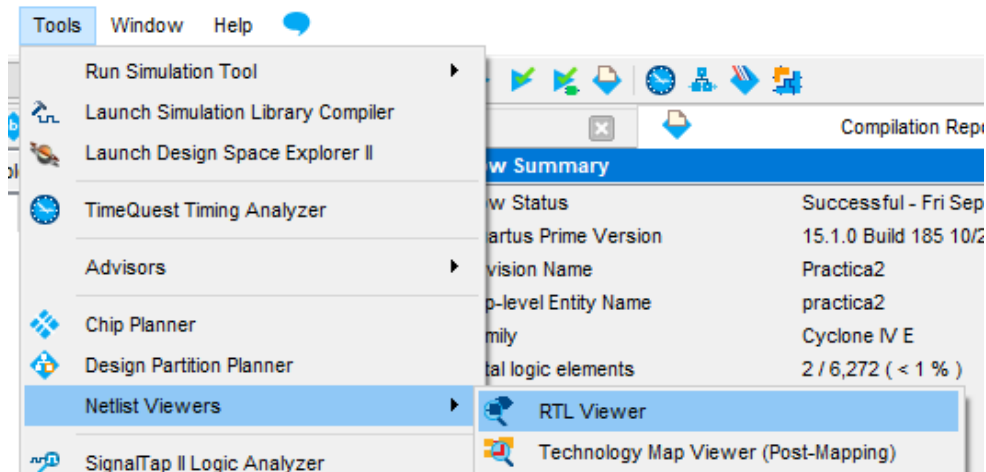


Imagen 13: RTL Viewer

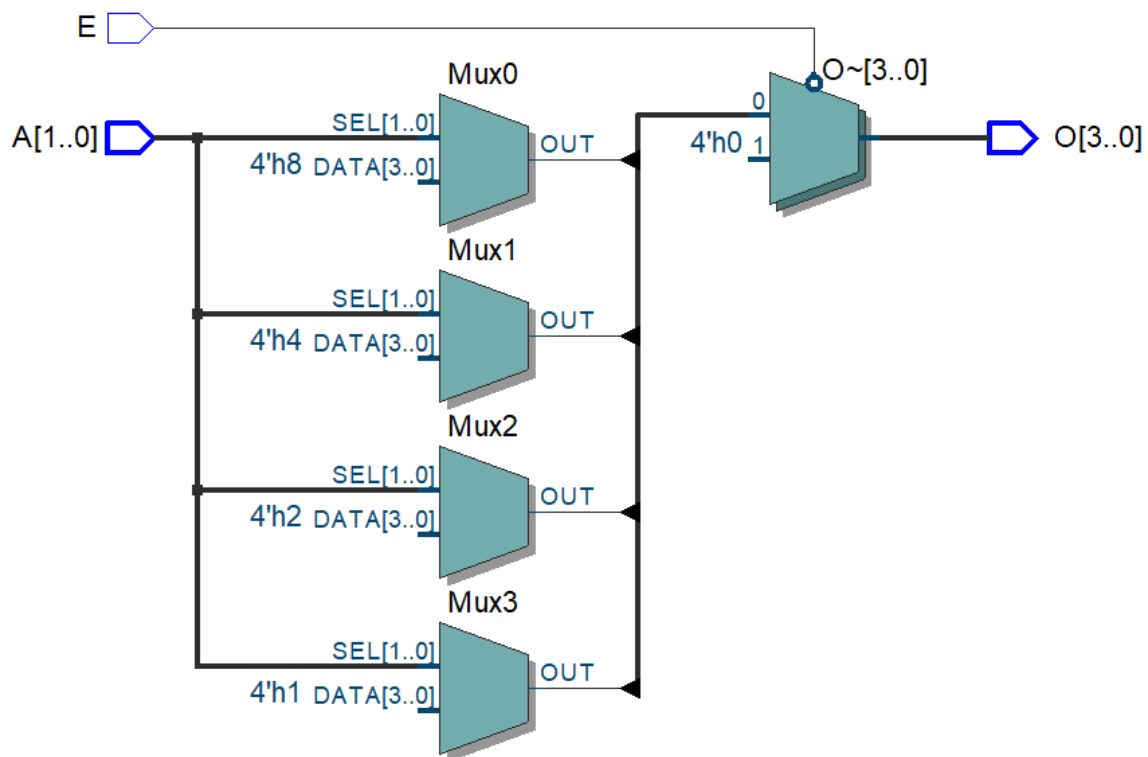


Imagen 14: Diagrama

7) Una compilación exitosa no garantiza un funcionamiento satisfactorio, pues podrían existir errores de lógica. Por lo anterior es conveniente simular comportamiento del circuito. Para ello se procederá a realizar la simulación de este

Simulación 1: Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso. En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.

Entradas:

- A(1) y A(0) → señales de dirección.
- E → señal de habilitación.

Salida:

- O(3 downto 0) → salidas decodificadas.

Todas las señales pueden ser probadas con un **reloj (clock)** de 20 ns, donde cada combinación cambia cada cierto ciclo.

Resultados esperados:

E	A1	A0	Salida O (O3 O2 O1 O0)	Descripción
0	X	X	0000	Deshabilitado (todas las salidas en 0)
1	0	0	0001	Activa salida O0
1	0	1	0010	Activa salida O1
1	1	0	0100	Activa salida O2
1	1	1	1000	Activa salida O3

Tabla 1: Resultados esperados

En el simulador asigno a A0 el siguiente valor:

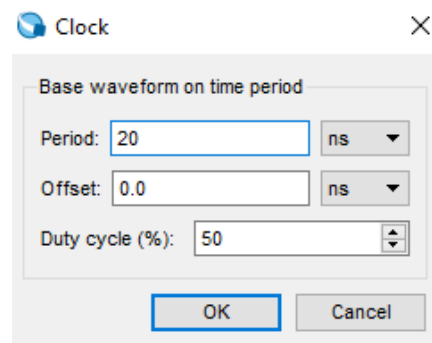


Imagen 15: valor A0

En el simulador asigno a A1 el siguiente valor:

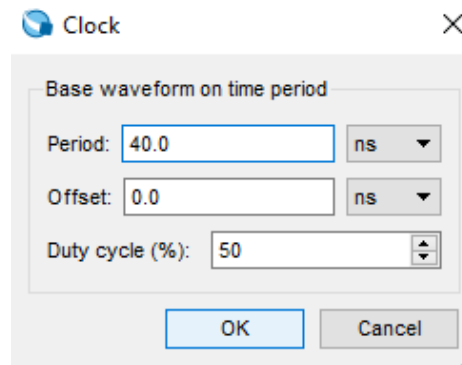


Imagen 16: valor A1

Los resultados son los siguientes con E=0:

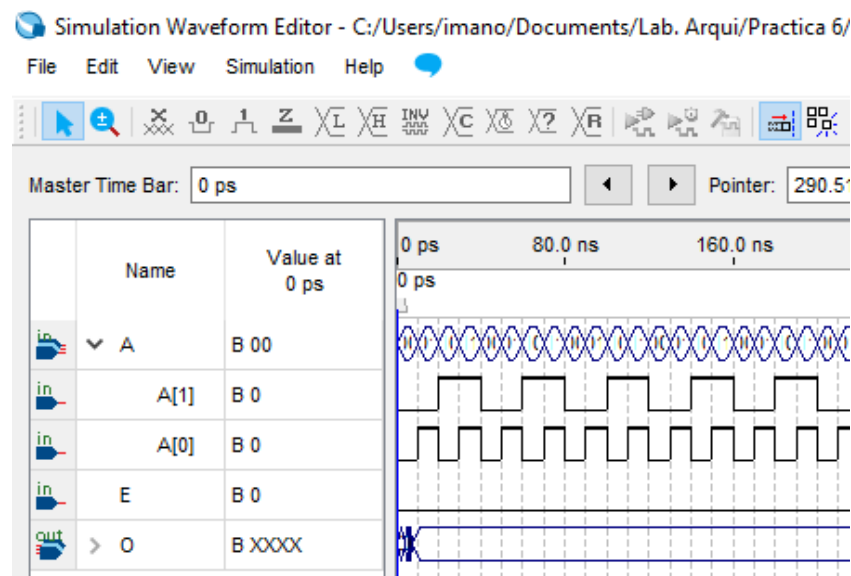


Imagen 17: Resultados con E=0

Cuando la entrada **E = '0'**, el bloque **no decodifica las entradas A**, sin importar los valores de A(1) y A(0).

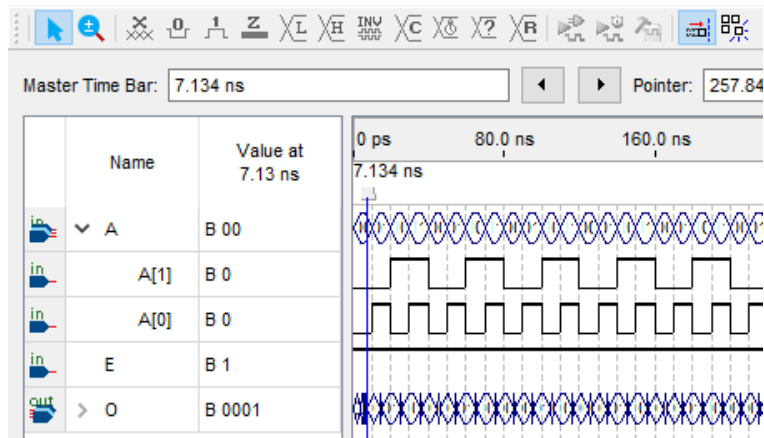
Interpretación:

- El decodificador está **inactivo o deshabilitado**.
- Todas las salidas permanecen en **'0'** (apagadas).
- Es como si el circuito estuviera “en espera” hasta que se active la señal E.

Los resultados son los siguientes con E=1:

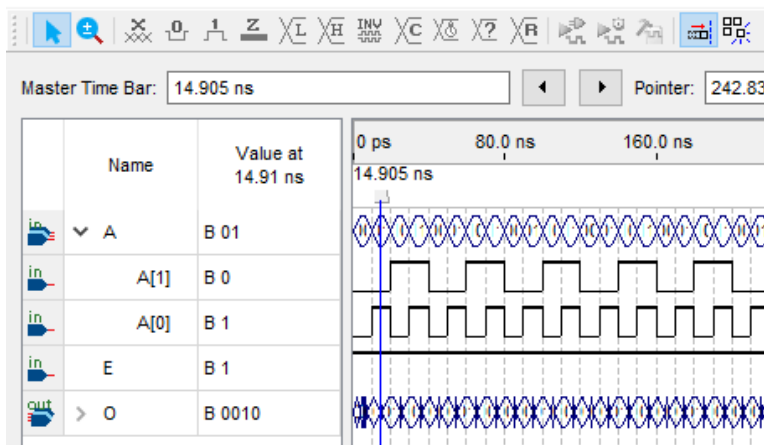
Simulation Waveform Editor - C:/Users/imano/Documents/Lab. Arqui/Practica 6/

File Edit View Simulation Help



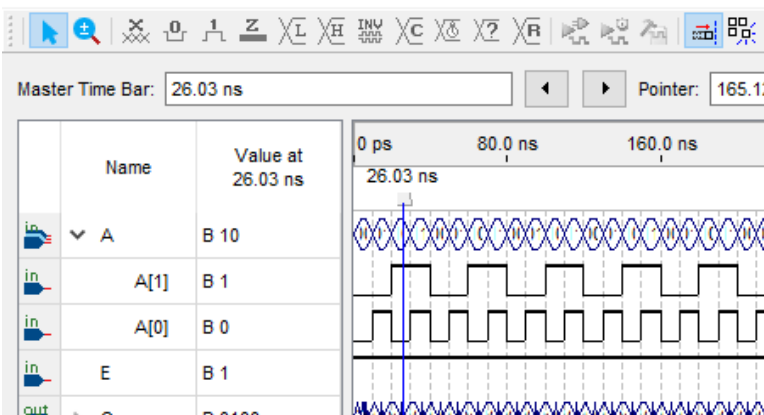
Simulation Waveform Editor - C:/Users/imano/Documents/Lab. Arqui/Practica 6/

File Edit View Simulation Help



Simulation Waveform Editor - C:/Users/imano/Documents/Lab. Arqui/Practica 6/

File Edit View Simulation Help



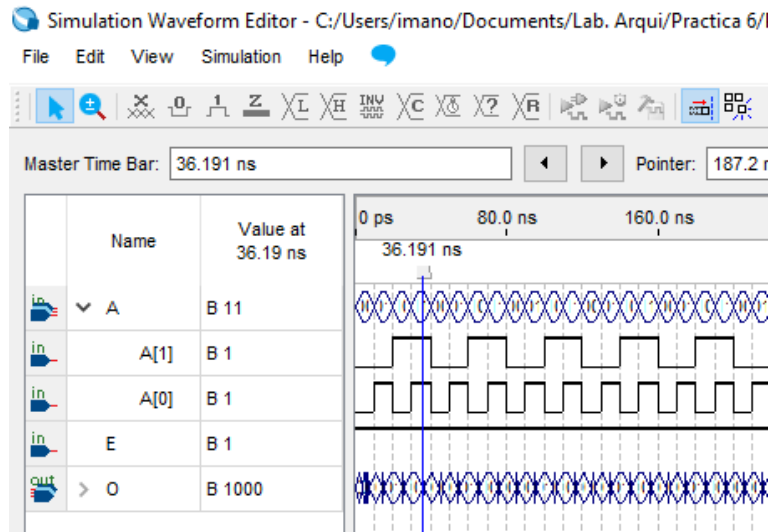


Imagen 18: Resultados con E=1

Cuando la entrada **E = '1'**, el circuito entra en modo activo y comienza a **decodificar** los bits de dirección **A(1:0)**. El bloque case A is se evalúa y selecciona **una sola salida** en alto ('1'), dependiendo de la combinación binaria de A.

Interpretación:

- Solo una salida de O(3 downto 0) estará en '1' a la vez.
- El bit activado representa la dirección binaria de A.

Simulación 2

- Utilizamos la otra simulación que es la misma interpretación

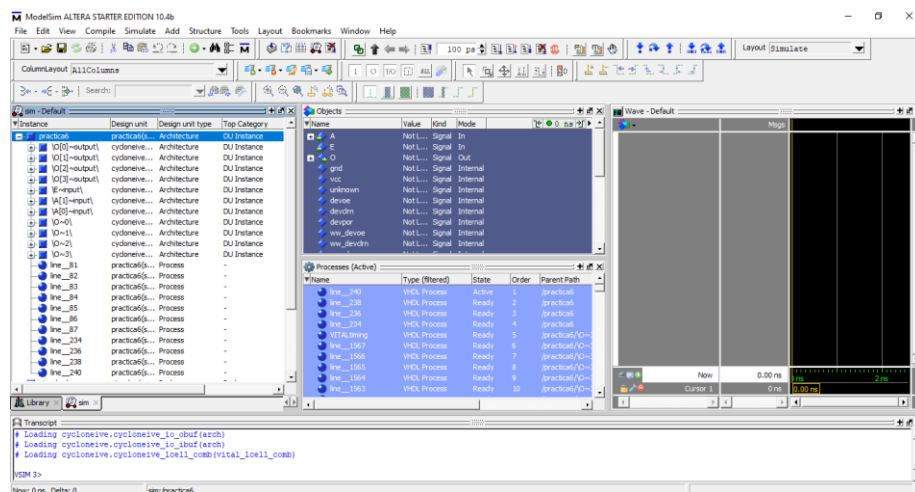
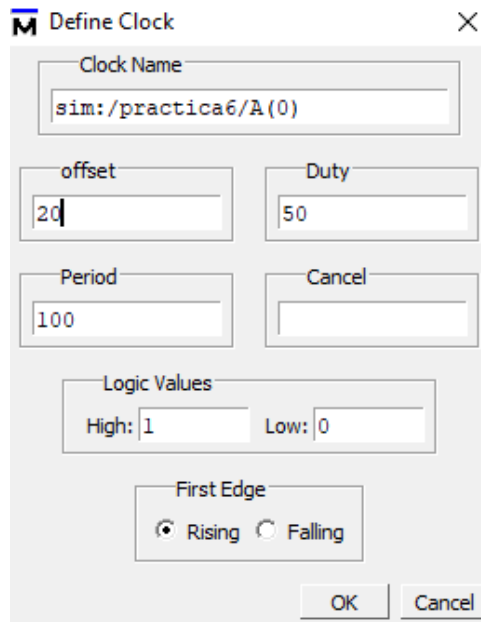


Imagen 19: ModelSim – Altera.

En el simulador asigno a A0 el siguiente valor:

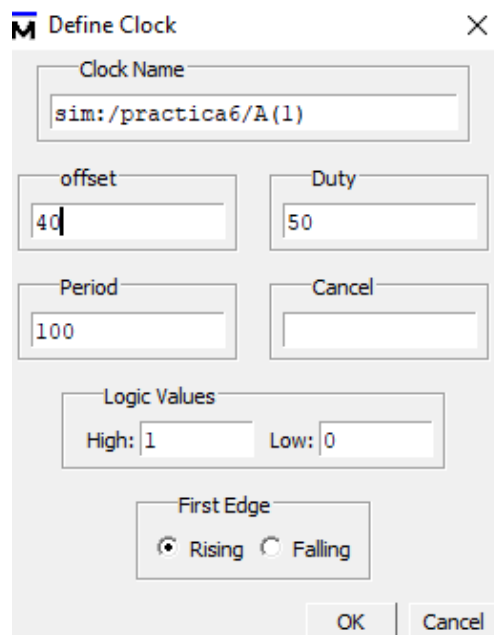


The image shows a 'Define Clock' dialog box with the following fields and settings:

- Clock Name:** `sim:/practica6/A(0)`
- offset:** `20`
- Duty:** `50`
- Period:** `100`
- Cancel:** (empty field)
- Logic Values:**
 - High:** `1`
 - Low:** `0`
- First Edge:** ☒ Rising ☐ Falling
- Buttons:** OK, Cancel

Imagen 20: valor A0

En el simulador asigno a A1 el siguiente valor:



The image shows a 'Define Clock' dialog box with the following fields and settings:

- Clock Name:** `sim:/practica6/A(1)`
- offset:** `40`
- Duty:** `50`
- Period:** `100`
- Cancel:** (empty field)
- Logic Values:**
 - High:** `1`
 - Low:** `0`
- First Edge:** ☒ Rising ☐ Falling
- Buttons:** OK, Cancel

Imagen 21: valor A1

Los resultados son los siguientes con $E=0$:

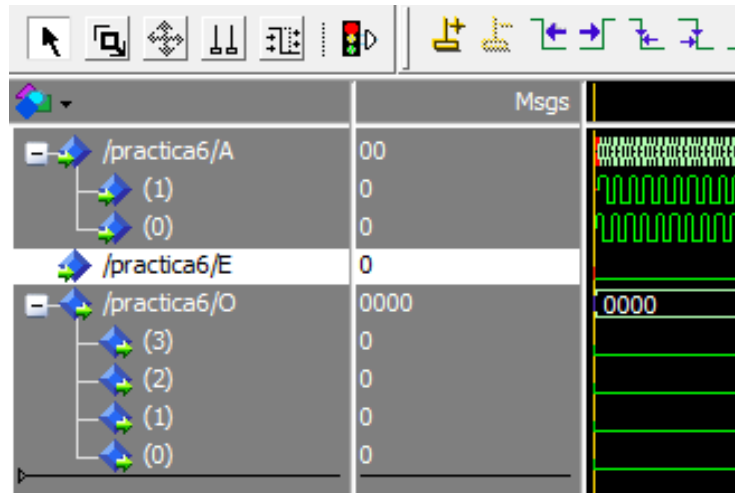


Imagen 22: Resultados con valor E=0

Los resultados son los siguientes con E=1:

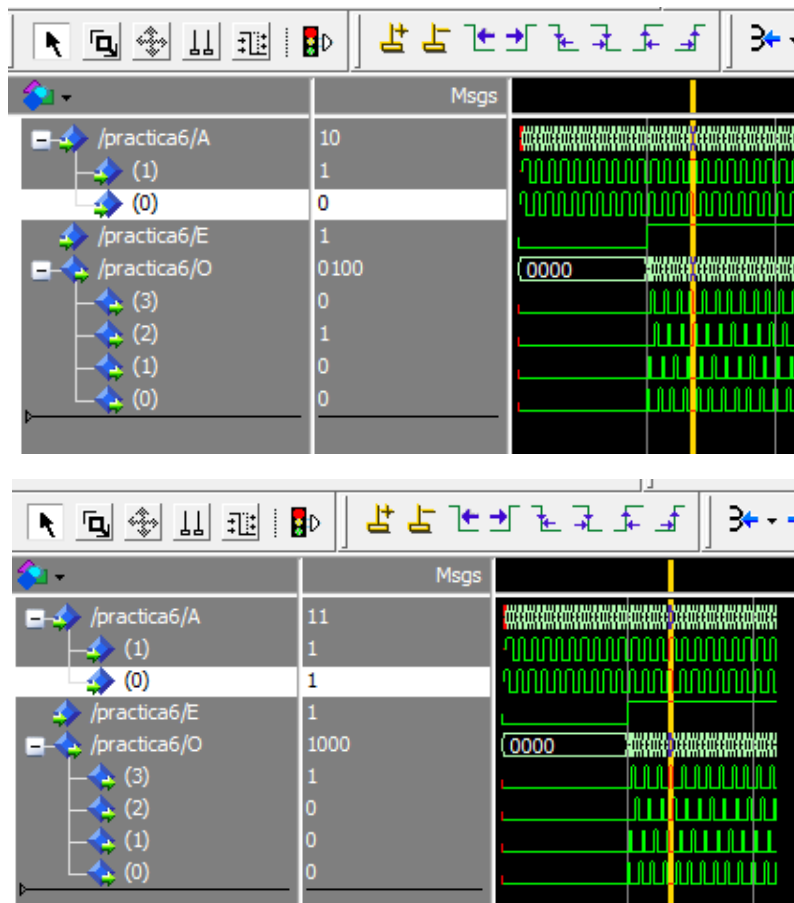


Imagen 23: Resultados con valor E=1

Con $E = 1$, el decodificador **interpreta la dirección binaria** de A y **activa únicamente una línea de salida**, lo que confirma su correcto funcionamiento como **decodificador 2 a 4**.

E	A1	A0	O3 O2 O1 O0	Interpretación
1	0	0	0001	Activa la salida O0
1	0	1	0010	Activa la salida O1
1	1	0	0100	Activa la salida O2
1	1	1	1000	Activa la salida O3

4. CONCLUSIÓN

Se validó con éxito el diseño y la síntesis de un decodificador 2 a 4, confirmando mediante simulación que la lógica implementada responde con precisión a la tabla de verdad teórica. El análisis demostró la criticidad de la señal de habilitación (E): se verificó que un estado $E = 0$ fuerza una condición de reposo en todas las salidas, mientras que un estado $E = 1$ permite la transferencia selectiva de datos hacia una única línea activa. La integración de Quartus Prime y ModelSim fue determinante para cerrar la brecha entre la descripción en VHDL y el comportamiento temporal del hardware. Esta experiencia no solo reafirmó el funcionamiento de los decodificadores en sistemas de direccionamiento, sino que subrayó la importancia de la simulación pre-síntesis como un paso obligatorio para garantizar la integridad del diseño antes de una eventual implementación en hardware físico (FPGA).

5. REFERENCIAS

1. Intel. (2024). *Using the Quartus Prime software for FPGA design*. Recuperado el 9 de mayo de 2025, de <https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html>
2. FPGA4Student. (2023). *VHDL tutorial: Multiplexer design using VHDL*. Recuperado el 9 de mayo de 2025, de <https://www.fpga4student.com/2017/06/vhdl-code-for-2-to-1-multiplexer.html>
3. AllAboutCircuits. (2024). *Introduction to multiplexers and demultiplexers*. Recuperado el 9 de mayo de 2025, de <https://www.allaboutcircuits.com/technical-articles/introduction-to-multiplexers-and-demultiplexers/>